

SageMath: Further Explorations in Graph Theory

We have now arrived at the fifth article in our series on SageMath. Recall that SageMath follows Python 3 syntax. Thus, effectively, SageMath is Python on steroids, offering all the power of Python with the additional kick from SageMath specific methods and utilities. We ended the last article in this series with a discussion of directed graphs, yet another variant of graphs that is extremely useful in handling many real-world scenarios. We will begin this article by continuing our discussion on directed graphs and then move on to directed multigraphs.



Before we begin, let us answer the two questions I posed in the last article in this series. The first question: How many edges does a complete directed graph on 10 vertices have? First, let us find the answer mathematically. There are 10 vertices, and each vertex has a directed edge to the 9 other vertices. Notice that, for each pair of vertices, there are two directed edges (one in each direction). Thus, the total number of directed edges in a complete directed graph on 10 vertices is $10 \times 9 = 90$. Now, let us verify this answer by executing SageMath code. Upon executing the following line of code on CoCalc, we get the answer as 90, the same as the one we found mathematically.

```
digraphs.Complete(10).size( )
```

Now, the second question: How many edges are there in an undirected complete graph with 10 vertices? First, let us find the answer mathematically. There are 10 vertices, and each vertex can connect to 9 other vertices. However,

since each edge is counted twice (once from each vertex of the pair), we divide by 2 to get the correct number of unique edges. Hence, the number of edges in an undirected complete graph on 10 vertices is $(10 \times 9) / 2 = 45$. Now, let us verify this answer by executing SageMath code. Upon executing the following line of code on CoCalc, we get the answer as 45, the same as the one we found mathematically.

```
graphs.CompleteGraph(10).size( )
```

Now, let us generate a slightly more complex directed graph for detailed analysis. The program 'dgraph12.sagews' shown below generates a directed graph called *Dgraph_12*.

```
DG = DiGraph( )
DG.add_vertices([0, 1, 2, 3, 4, 5])
DG.add_edges([(0, 1), (0, 4), (1, 2), (1, 4), (2, 3), (3, 1),
(3, 4), (4, 5), (5, 0)])
DG.show( )
```